

2026.03.24.화

🕒 생성일	@2026년 3월 24일 오전 9:03
🏷 태그	Linux

목차

1. 리눅스 명령어
2. 리눅스 문서 편집기
3. 셸 스크립트

▼ 1) 리눅스 명령어

- 경로 이동: cd
- 파일 복사: cp
- 디렉터리 생성: mkdir
- 파일명 변경, 경로 이동: mv
- 매뉴얼, 도움말: man
- 명령어 실행 파일 위치: which
- 매뉴얼이나 소스 코드 위치: whereis
- 사용자 추가: sudo adduser
- 비밀번호 변경: sudo passwd
- 사용자 삭제: sudo deluser
- 권한 변경: chmod
- 아카이빙: tar

```
# 아카이빙
tar -cf test .tar [파일명1] [파일명2]

# 해제
tar -xf test.tar
```

- 압축: gzip, bzip2, xz, zip

구분	gzip	bzip2	xz	zip
압축률	보통	높음	매우 높음	보통
압축 속도	빠름	느림	매우 느림	보통
압축 해제 속도	빠름	느림	느림	빠름
포맷	.gz	.bz2	.xz	.zip
여러 파일 압축	불가	불가	불가	가능
장점	속도 빠름	압축률 좋음	최고의 압축률	윈도우 호환성
단점	압축률 낮음	느림	매우 느림	낮은 압축률

```
압축 : [명령어] [압축할 파일명]
압축 : zip [압축된 파일명] .zip [압축할 파일명]
```

해제 : [명령어] -d [압축 해제할 파일명]
 해제 : unzip [압축 해제할 파일명]

- 모든 프로세스 정보 출력: ps -ef
- 특정 프로세스 검색: pgrep
- 실시간 프로세스 정보 모니터링: top
- 프로세스 정보 보내기: kill [옵션] [PID]

신호 이름	번호	설명
SIGTERM	15	정상적인 종료 요청 (기본값)
SIGKILL	9	강제 종료 (정상 종료가 되지 않을 때 사용)
SIGSTOP	19	일시 정지
SIGCONT	18	정지된 프로세스 재개

▼ 2) 리눅스 문서 편집기

- 문서 내용 출력: cat
- 문서 처음 10줄 출력: head [파일명]
- 문서 마지막 10줄 출력: tail [파일명]
- 리다이렉션 - 출력 내용을 파일로

구분	기호	설명	예시
표준 입력	<	표준 입력을 파일에서 읽음	sort < fruits.txt
표준 출력	>	표준 출력을 파일에 저장 (덮어쓰기)	ls -l > ls.txt
	>>	표준 출력을 파일에 추가	ls -l >> ls.txt
표준 에러	2>	표준 에러를 파일에 저장 (덮어쓰기)	mkdir 2> error.txt
	2>>	표준 에러를 파일에 추가	mkdir 2>> error.txt
표준 출력	&>	표준 출력과 에러를 파일에 저장 (덮어쓰기)	[명령어] &> outout.txt
표준 에러	&>>	표준 출력과 에러를 파일에 추가	[명령어] &>> outout.txt

- nano

표시	단축키	설명
^G	[Ctrl] + [G]	도움말 메뉴를 표시하는 단축키
^O	[Ctrl] + [O]	파일을 저장하는 단축키로 저장할 파일의 이름을 묻는 프롬프트가 표시됨
^W	[Ctrl] + [W]	파일 내에서 문자열을 검색하는 단축키로 검색할 문자열을 입력하고 <엔터>를 누름
^K	[Ctrl] + [K]	현재 커서가 있는 줄 또는 선택한 텍스트를 잘라내는 단축키
^T	[Ctrl] + [T]	문자 편집기 내에서 명령어 실행 결과를 출력하는 단축키
^C	[Ctrl] + [C]	입력한 전체 문자의 행과 열, 문자의 길이 대비 현재 커서의 위치를 나타내는 단축키
^X	[Ctrl] + [X]	편집기를 종료하는 단축키로, 변경 사항이 있을 경우 저장 여부를 묻는 프롬프트 표시
^R	[Ctrl] + [R]	파일로부터 읽은 문자를 현재 문서 편집기에 덧붙이는 단축키
^W	[Ctrl] + [W]	특정 문자를 다른 문자로 변경하는 단축키
^U	[Ctrl] + [U]	잘라내거나 복사한 문자를 붙여넣기 하는 단축키
^J	[Ctrl] + [J]	문자 정렬을 통해 정돈된 형태로 변경할 때 사용하는 단축키
^/	[Ctrl] + [/]	문서 내 특정 행과 열로 이동할 수 있는 단축키

- vi

문자	의미
i	커서 앞에 문자 입력
I	커서가 있는 첫 줄에 문자 입력
a	커서 뒤에 문자 입력
A	커서가 있는 끝 줄에 문자 입력
o	커서가 있는 줄 아래에 문자 입력
O	커서가 있는 줄 위에 문자 입력

- sed

```
# 1번줄부터 끝까지 출력
sed -n -e '1, $p' sample

# 1번줄부터 3번 줄까지 출력
sed -n -e '1, 3p' sample

# 1번줄과 4번 줄 출력
sed -n -e '1p' -e '4p' sample

# 특정 문자열을 포함한 줄 출력
sed -n -e '/1111/p' sample
```

```
# 1번부터 끝까지 중 2번 줄부터 4번 줄 제거
sed -n -e '2,4d' -e '1,$p' sample

# 특정 문자 변경하여 출력
sed -n -e 's/Pak/PAK/g' -e '1,$p' sample

# 대소문자 구분없이 특정 문자 변경하여 출력
sed -n -e 's/Pak/PAK/gi' -e '1,$p' sample

# 특정 문자열 뒤에 다른 파일 내용 추가
sed -n -e '/4444$/r add' -e '1,$p' sample
```

- awk

행과 열 구조의 데이터 처리에 최적화

```
# 1번 열 출력
awk '{ print $0 }' sample

# 1번, 3번 열 출력
awk '{ print $0, $2 }' sample

# 문자 붙여서 출력
awk '{ print "Name:"$1, "Num:"$2 }' sample

# 특정 패턴 추가
awk '$2 == 10 { print $2 }' sample
awk '$2 >= 20 { print $2 }' sample
awk '$2 < 10 { print $2 }' sample
awk '$2 != 30 { print $2 }' sample

# 문서 내용에 없는 변수 사용한 값 출력
awk '{sum += $3} END { print "SUM:" sum }' sample
awk '{sum += $3} { print "SUM:"sum }' sample
```

▼ 3) 셸 스크립트

- 현재 셸 확인: echo \$SHELL
- 셸에 변수 저장: 변수명=값
- 변수에 저장된 값 출력: echo \$변수명
- 변수 선언 값 제거: unset 변수명
- 환경 변수

환경 변수	설명	출력 예시
PATH	실행 파일을 찾는 경로 목록	/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:
HOME	현재 사용자의 홈 디렉터리	/home/ubuntu
USER	현재 로그인한 사용자의 이름	ubuntu
LANG	시스템 언어 설정	en_US.UTF-8
SHELL	사용 중인 기본 로그인 셸 경로	/bin/bash

- 환경 변수 등록: export 변수명=값
- 그대로 출력: echo
- 셸 스크립트는 첫 줄에 셸을 지정해주어야함

```
#!/bin/bash
echo "Hello, Linux World!!"

# 재부팅이나 다른 환경에서도 실행하기 위해 현재 사용자의 홈 디렉터리를 환경 변수에 추가
export PATH=$PATH:/home/username/bin
```

- 셸 스크립트 위치 매개변수

위치 매개변수	설명
\$0	실행된 스크립트의 이름
\$1	첫 번째 인자
\$2 ~ \$9	두 번째 인자 ~ 아홉 번째 인자
\${10} ~	열 번째 인자 (두자수 이상의 인자는 중괄호 필요)
\$#	전달된 인자의 개수
\$@	전달된 모든 인자 (개별적으로 처리)
\$*	전달된 모든 인자 (하나의 문자열로 처리)

```
=====args.sh=====
#!/bin/bash

echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "Arguments number: $#"
```

```
echo "All arguments: $@"
echo "All arguments: $*"
=====
> args.sh apple banana cherry
> Script name: /home/username/bin/args.sh
> First argument: apple
> Second argument: banana
> Arguments number: 3
> All arguments: apple banana cherry
> All arguments: apple banana cherry
```

- 문자열 형식

% 문자	설명	예시 (입력 → 출력)
%s	문자열	"가나다" → 가나다
%d	10진수 정수	75 → 75
%f	소수점을 포함한 실수	3.1415 → 3.1415
%.2f	소수점 2자리까지의 실수	3.1415 → 3.14
%x	16진수(소문자), %X는 대문자	255 → ff
%o	8진수	8 → 10

- 조건문

```
#!/bin/bash

if test -e file.txt; then
```

```
    echo "The file exists"
fi
```

```
#!/bin/bash
```

```
if test -e file.txt; then
    echo "The file exists"
else
    echo "The file does not exists"
fi
```

- test 명령어

유형	옵션 / 조건식	설명
파일	-e [파일명]	파일이나 디렉터리가 존재하면 참, 그렇지 않으면 거짓
	-f [파일명]	파일이 존재하고, 그 파일이 일반 파일이면 참, 그렇지 않으면 거짓
	-d [파일명]	파일이 존재하고, 그 파일이 디렉터리이면 참, 그렇지 않으면 거짓
	-r [파일명]	파일이 존재하고, 그 파일이 읽기 가능하면 참, 그렇지 않으면 거짓
	-w [파일명]	파일이 존재하고, 그 파일이 쓰기 가능하면 참, 그렇지 않으면 거짓
	-x [파일명]	파일이 존재하고, 그 파일이 실행 가능하면 참, 그렇지 않으면 거짓
문자열	str1= str2	두 문자열이 같으면 참, 그렇지 않으면 거짓
	str1 != str2	두 문자열이 다르면 참, 그렇지 않으면 거짓
	-z str	문자열의 길이가 0이면 참, 그렇지 않으면 거짓
	-n str	문자열의 길이가 0이 아니면 참, 그렇지 않으면 거짓
숫자	n1 -eq n2	두 숫자가 같으면 참, 다르면 거짓
	n1 -ne n2	두 숫자가 다르면 참, 같으면 거짓
	n1 -gt n2	앞의 숫자가 뒤의 숫자보다 크면 참, 작거나 같으면 거짓 (n1 > n2)
	n1 -lt n2	앞의 숫자가 뒤의 숫자보다 작으면 참, 크거나 같으면 거짓 (n1 < n2)
	n1 -ge n2	앞의 숫자가 뒤의 숫자보다 크거나 같으면 참, 작으면 거짓 (n1 >= n2)
	n1 -le n2	앞의 숫자가 뒤의 숫자보다 작거나 같으면 참, 크면 거짓 (n1 <= n2)

```
case "$변수명" in
패턴1)
```

```

        작업1 ;;
패턴2)
        작업2 ;;
*)
        패턴1과 패턴2가 아닐 때 수행할 작업 ;;
esac

```

```

#!/bin/bash

echo "1) Shutdown the system"
echo "2) Restart the system"
echo -n "what do you want? (1/2): "
read answer
case $answer in
    1) echo "Shutdown the system" ;;
    2) echo "Restart the system" ;;
    *) echo "Wrong command" ;;
esac

```

- 반복문

```

for 변수 in 리스트
do
    명령어
done

```

```

#!/bin/bash

for name in Kim Lee Pak
do
    echo "Hello, $name!"
done

```

```

#!/bin/bash

for i in {1..10}
do
    echo "Count: $i"
done

```

```

#!/bin/bash

for ((i=1; i<=10; i+=2))
do
    echo "Step: $i"
done

```

```

#!/bin/bash

num=1
while ((num <= 5))
do
    echo "Number: $num"
    ((num++))
done

```

```
#!/bin/bash
```

```
num=1
until ((num > 5))
do
    echo "Number: $num"
    ((num++))
done
```

```
#!/bin/bash
```

```
num=1
while ((num <= 10 ))
do
    if ((num % 2 == 0)); then
        echo $num "is an even"
    else
        echo $num "is an add"
    fi
    ((num++))
done
```